

Computer-Assisted Formalization of Mathematics

Week 04 - Inductive types and induction

First Half

- ▶ Constructors and recursive data
- ▶ Natural numbers
- ▶ Lists and length-indexed vectors

Second Half

- ▶ Constructor equalities
- ▶ Pattern matching and recursors
- ▶ Structural and functional induction

Jaume de Dios Pont - jdedios@nyu.edu

<https://nyulean26.github.io/>

Inductive Types

Tools of inductive types

Induction

Sum types again

- ▶ A sum type can have multiple *constructors* (`Or.inl`, `Or.inr`)
- ▶ For example, one constructor can tag each kind of number.

```
inductive Number : Type where
  | fromNat : Nat → Number
  | fromReal : Real → Number
  | fromComplex : Complex → Number
```

- ▶ Inductive types are defined *tautologically* by their constructors.

```
inductive Boolean : Type where
  | true : Boolean
  | false : Boolean
```

Inductive types defined recursively

Inductive types can be used **recursively** in their constructors!

Recursive example 1: Natural Numbers

- Defined inductively with two constructors:

```
inductive MyNat : Type where
  | zero : MyNat
  | succ : MyNat → MyNat
```

- zero is the base case.
- succ defines the next natural number.

$$\mathbb{N} := \{\text{zero}, \text{succ zero}, \text{succ (succ zero)}, \dots\}$$

Lists

Constructors can take more than one argument.

Recursive example 2: Lists of elements of type $\mathbb{N}$

```
inductive NatList : Type where
  | nil : NatList
  | cons : Nat → NatList → NatList
```

► `nil` is the empty list; `cons` adds an element to a list.

```
def exampleList : NatList :=
  NatList.cons 2 (NatList.cons 3 (NatList.cons 4 NatList.nil))
```

Lists

Inductive types can be parametrized by other types.

Recursive example 2': Lists of elements of type α

```
inductive MyList ( $\alpha$  : Type) : Type where  
  | nil : MyList  $\alpha$   
  | cons :  $\alpha \rightarrow$  MyList  $\alpha \rightarrow$  MyList  $\alpha$ 
```

- ▶ `nil` is the empty list; `cons` adds an element to a list.
- ▶ The type `MyList  $\alpha$` records the element type, but not the length.

Vectors

Constructors can be indexed by **the value** other types.

Recursive example 3: Lists indexed by their length

```
inductive Vec ( $\alpha$  : Type) : Nat  $\rightarrow$  Type where
| nil : Vec  $\alpha$  0
| cons : {n : Nat}  $\rightarrow$   $\alpha \rightarrow$  Vec  $\alpha$  n  $\rightarrow$  Vec  $\alpha$  (n + 1)
```

- ▶ nil has length 0; cons turns a length- n vector into a length- $(n + 1)$ vector.
- ▶ The length n is implicit because Lean can infer it from the tail.

Inductive Types

Tools of inductive types

Induction

The no-confusion principle

- Different constructors are disjoint:

$$\text{Nat.zero} \neq \text{Nat.succ } n$$

- The same constructor is injective:

$$\text{Nat.succ } m = \text{Nat.succ } n \implies m = n$$

Lean generates a `Nat.noConfusion` lemma automatically from the inductive definition. Tactics can use it semi-automatically.

Recursive functions in practice: `match`

We usually write the constructor cases directly:

```
def double (n : Nat) : Nat :=  
  match n with  
  | 0      => 0  
  | k + 1 => double k + 2
```

- ▶ Patterns must cover every possible input. The second branch names the predecessor k .
- ▶ The recursive call is on k , which is **structurally smaller**. Lean checks termination.
- ▶ `match` can also sit inside a larger expression. Lean compiles these cases to recursors for the kernel to check.

Case analysis: `Nat.casesOn`

Every natural number is either zero or `succ k`. To define $f(n)$, give a result for each case.

$$\begin{array}{ll} n = 0 & \longmapsto z \\ n = \text{Nat.succ } k & \longmapsto s\ k \end{array}$$

$$f(n) = \text{Nat.casesOn } n\ z\ s$$

Here z is the zero result; s uses the predecessor k .

Example: predecessor sends 0 to 0 and `Nat.succ k` to k . The successor case does not receive a result for k .

Recursion and induction: `Nat.rec`

The recursor adds one piece of information to the successor case: the result already obtained for k .

$$\begin{aligned} n = 0 & \longmapsto z \\ n = \text{Nat.succ } k & \longmapsto s \ k \ (f(k)) \\ f(n) &= \text{Nat.rec } z \ s \ n \end{aligned}$$

Computing: to double a number, use $z = 0$ and $s \ k \ r = r + 2$.

Proving: for a goal $P(n)$, the successor case receives a proof of $P(k)$: the induction hypothesis.

Inductive Types

Tools of inductive types

Induction

Induction is recursion into Prop

Start with a **proposition-valued function** $P : \mathbb{N} \rightarrow \text{Prop}$. Induction constructs a function assigning a proof to every input:

$$f : (n : \mathbb{N}) \rightarrow P(n).$$

Base proof: $h_0 : P(0)$

Step: $h_s : (k : \mathbb{N}) \rightarrow P(k) \rightarrow P(k + 1)$

$$f(n) := \text{Nat.recOn } n \ h_0 \ h_s : P(n).$$

The recursive result in the step is a proof of $P(k)$: the **induction hypothesis**.

`Nat.recOn` is `Nat.rec` with n supplied first.

Structural induction on lists

The same rule follows the constructors of `List α`.

Empty list: prove $P(\text{nil})$

Cons: from $P(xs)$, prove $P(\text{cons } a \ xs)$

The hypothesis concerns the **tail** xs , the recursive argument of `cons`.

Example: prove `append xs nil = xs`. In the `cons` case, unfolding `append` and comparing both heads leaves the induction hypothesis for the tail.

Functional induction

Sometimes the useful cases come from a **function's definition**, not directly from the constructors of its input.

```
def half : Nat → Nat
| 0 => 0
| 1 => 0
| n + 2 => half n + 1
```


Functional induction

Sometimes the useful cases come from a **function's definition**, not directly from the constructors of its input.

```
def half : Nat → Nat
| 0 => 0
| 1 => 0
| n + 2 => half n + 1
```

`fun_induction half` follows these three equations. In the recursive case it supplies

$$\text{half } n \leq n$$

as the induction hypothesis for proving `half (n + 2) ≤ n + 2`.

Functional induction

Sometimes the useful cases come from a **function's definition**, not directly from the constructors of its input.

```
def half : Nat → Nat
| 0 => 0
| 1 => 0
| n + 2 => half n + 1
```

`fun_induction half` follows these three equations. In the recursive case it supplies

$$\text{half } n \leq n$$

as the induction hypothesis for proving $\text{half } (n + 2) \leq n + 2$.

```
theorem half_le (n : Nat) : half n ≤ n := by
  fun_induction half <;> omega
```

Once the recursion supplies the right hypothesis, the rest is arithmetic.

More advanced forms of induction

The `Nat.rec` step we used computes $f(k + 1)$ from one result, $f(k)$. Fibonacci's usual equation asks for **two** earlier values:

$$F(n + 2) = F(n + 1) + F(n).$$

- ▶ This equation does not fit that simple one-result step directly.
- ▶ `match n with still accepts` is as **structural recursion**: both calls have smaller inputs.
- ▶ For calls that decrease by a measure rather than by constructor structure, use **well-founded recursion and induction**.
Example: Induction in $\mathbb{N}^d$ using the lexicographic order.